

Sistema Web Gaby Salón y Spa

Documentación TÉCNICA completa

Explicación profunda de la arquitectura, el código y el funcionamiento interno. Cada concepto se explica desde cero: no se asume conocimiento previo de programación, frameworks, librerías ni Firebase.

Next.js · React · TypeScript · Tailwind CSS · Firebase (Auth + Firestore)

Índice

1. Introducción y visión general de la arquitectura
2. Conceptos fundamentales explicados desde cero
3. Arquitectura por capas del sistema
4. Estructura completa de archivos
5. Archivos de configuración
6. Punto de entrada: layout y página principal
7. Sistema de rutas de Next.js
8. Capa de conexión a Firebase (firebase.ts)
9. Capa de datos: servicios (services.ts)
10. Capa de datos: registro diario (records.ts)
11. Capa de datos: reservas y control de cupos (booking.ts)
12. Modelo de datos en Firestore
13. Autenticación y protección de rutas
14. Componentes del sitio público
15. Componentes reutilizables
16. Módulos del panel de administración
17. Sistema de estilos (Tailwind y CSS global)
18. Reglas de seguridad de Firestore
19. Ciclo de vida de un dato (flujo técnico)
20. Glosario técnico
21. Cómo extender el proyecto (agregar un módulo)

1. Introducción y visión general

Este sistema es una **aplicación web** construida con **Next.js** (un framework de React) y **Firebase** (servicios de backend de Google). Tiene dos frentes: un **sitio público** informativo con reservas, y un **panel de administración** protegido por login.

La idea central de la arquitectura es la **separación en capas**:

- **Capa de presentación (UI)**: las páginas y componentes visuales (carpeta `src/app` y `src/components`).
- **Capa de datos (lógica de acceso)**: funciones que hablan con la base de datos (carpeta `src/lib`).
- **Capa de persistencia (backend)**: Firebase, que guarda los datos (Firestore) y valida usuarios (Authentication).

La UI **nunca** habla directamente con Firebase: siempre pasa por la capa de datos (`src/lib`).

Esto mantiene el código ordenado y fácil de mantener.

2. Conceptos fundamentales (desde cero)

Antes de ver el código, hay que entender las piezas. Cada una explicada sin asumir conocimiento previo.

2.1 HTML, CSS y JavaScript

Son los 3 lenguajes base de toda página web:

- **HTML:** define la **estructura** (títulos, párrafos, botones). Es el "esqueleto".
- **CSS:** define el **diseño** (colores, tamaños, posiciones). Es la "ropa".
- **JavaScript:** define el **comportamiento** (qué pasa al hacer clic, cargar datos). Es el "movimiento".

2.2 TypeScript

Definición: TypeScript es JavaScript con **tipos**. Un "tipo" indica qué clase de dato es una variable (texto, número, verdadero/falso).

Ejemplo: al escribir `price: number` le decimos al programa que `price` siempre será un número. Si por error se le asigna un texto, el editor avisa **antes** de ejecutar. Esto previene errores. Los archivos TypeScript terminan en `.ts` (o `.tsx` si contienen HTML).

2.3 React

Definición: React es una librería para construir interfaces dividiéndolas en **componentes**: piezas independientes y reutilizables.

Conceptos clave de React que aparecen en todo el proyecto:

Concepto	Qué es
Componente	Una función que devuelve la interfaz (con JSX). Ej: <code>Navbar()</code> .
JSX	Sintaxis que mezcla HTML dentro de JavaScript. Ej: <code><button>Reservar</button></code> .
Props	Los "parámetros" que recibe un componente para personalizarlo.
Estado (state)	Datos que cambian con el tiempo y hacen que la pantalla se redibuje. Se maneja con <code>useState</code> .
Hook	Función especial de React que empieza con "use". Ej: <code>useState</code> , <code>useEffect</code> .

useState guarda un valor que puede cambiar. **useEffect** ejecuta código cuando el componente aparece o cuando cambia algo (ej: cargar datos al abrir la pantalla).

```
// Ejemplo de estado: guardar el texto de un campo
const [email, setEmail] = useState("");
// email = valor actual; setEmail = función para cambiarlo
```

2.4 Next.js

Definición: Next.js es un **framework** construido sobre React que agrega organización de páginas por carpetas, optimizaciones y herramientas listas para usar.

Lo más importante que usa este proyecto:

- **App Router:** cada carpeta dentro de `src/app` con un archivo `page.tsx` se convierte en una **página con su propia dirección (URL)**. Ej: `src/app/reservar/page.tsx` → `/reservar`.
- **"use client":** una línea al inicio de un archivo que indica que ese componente corre en el navegador (necesario para usar estado, clics, etc.).

2.5 Tailwind CSS

Definición: Tailwind es una forma de escribir CSS usando **clases cortas** directamente en el HTML, en vez de archivos de estilos separados.

Ejemplo: `class="flex items-center gap-3"` significa "poné los elementos en fila, centrados verticalmente, con separación". Cada clase hace una cosa concreta.

2.6 Node.js y npm

- **Node.js:** permite ejecutar JavaScript fuera del navegador (necesario para desarrollar).
- **npm:** el administrador de paquetes. Con `npm install` descarga las librerías; con `npm run dev` arranca el proyecto.

2.7 Firebase

Definición: Firebase es un conjunto de servicios en la nube de Google. Este proyecto usa dos: **Authentication** (login) y **Cloud Firestore** (base de datos).

Firestore es una base de datos "NoSQL de documentos": en vez de tablas y filas (como SQL), guarda **colecciones** (grupos) que contienen **documentos** (fichas con campos tipo `clave: valor`). Es flexible y funciona en tiempo real.

3. Arquitectura por capas

Este es el recorrido que hace la información, de arriba (usuario) hacia abajo (base de datos):



Regla de oro del proyecto: los componentes de la UI llaman a funciones de `src/lib`, y solo esas funciones tocan Firebase. Así, si algún día se cambia la base de datos, solo hay que modificar `src/lib` y no todas las pantallas.

4. Estructura completa de archivos

```
GabySalaon/
├── public/images/           imágenes y fotos
├── src/
│   ├── app/
│   │   ├── layout.tsx      plantilla global (fuentes, metadatos)
│   │   ├── page.tsx        página principal (arma las secciones)
│   │   ├── globals.css     estilos globales y clases propias
│   │   ├── reservar/
│   │   │   └── page.tsx     formulario público de reservas
│   │   └── admin/
│   │       ├── page.tsx     panel principal
│   │       ├── login/page.tsx inicio de sesión
│   │       ├── services/page.tsx gestión de servicios
│   │       ├── reservations/page.tsx gestión de reservas
│   │       ├── records/page.tsx registro diario
│   │       └── reports/page.tsx reportes
│   ├── components/
│   │   ├── navbar.tsx, footer.tsx
│   │   ├── service-picker.tsx combobox de servicios
│   │   ├── whatsapp-icon.tsx, whatsapp-floating.tsx
│   │   ├── section-shell.tsx, utils.ts
│   │   └── sections/        secciones de la home
│   │       ├── hero.tsx, about.tsx, services.tsx
│   │       └── gallery.tsx, testimonials.tsx, contact.tsx
│   ├── content/
│   │   └── site.ts          datos del salón y configuración
│   └── lib/
│       ├── firebase.ts     conexión a Firebase
│       ├── services.ts     funciones de servicios
│       ├── records.ts       funciones de registro diario
│       └── booking.ts       funciones de reservas
├── firestore.rules          reglas de seguridad
├── .env.local               llaves privadas (no se sube)
├── package.json             dependencias y comandos
└── next.config.mjs, tsconfig.json, tailwind.config.ts
```

5. Archivos de configuración

package.json

Lista las **dependencias** (librerías que usa el proyecto) y los **comandos**:

```
// comandos disponibles
"dev": "next dev"    → arranca en modo desarrollo
"build": "next build" → compila para producción
"start": "next start" → corre la versión compilada
```

Dependencias principales: `next`, `react`, `firebase`, `framer-motion`, `lucide-react`, `tailwindcss`.

next.config.mjs

```
const nextConfig = { images: { unoptimized: true } };
```

Configura Next.js. `images: unoptimized` le dice que no procese las imágenes automáticamente (útil para exportar el sitio como archivos estáticos).

tsconfig.json

Configura TypeScript. Lo más relevante es el "alias" de rutas:

```
"paths": { "@/*": ["/src/*"] }
```

Esto permite escribir `@/lib/firebase` en vez de rutas largas como `../../lib/firebase`. El `@` significa "la carpeta src".

.env.local

Guarda las llaves de Firebase como **variables de entorno**. Todas empiezan con `NEXT_PUBLIC_`, lo que en Next.js significa que estarán disponibles en el navegador:

```
NEXT_PUBLIC_FIREBASE_API_KEY=...
NEXT_PUBLIC_FIREBASE_PROJECT_ID=...
(y las demás llaves)
```

Seguridad: este archivo está listado en `.gitignore`, por eso nunca se sube a GitHub.

6. Punto de entrada: layout y página principal

src/app/layout.tsx

Es la **plantilla** que envuelve a todas las páginas. Define el idioma, las fuentes tipográficas y los **metadatos** (título y descripción que ve Google y las redes sociales).

```
// carga dos fuentes de Google
const inter = Inter({ subsets: ["latin"] });
const playfair = Playfair_Display({ subsets: ["latin"] });

export const metadata = {
  title: "Gaby Salón y Spa | ...",
  description: "Salón y spa profesional en Alajuela...",
};
```

src/app/page.tsx

Es la **página principal**. Solo arma las secciones en orden, como piezas de lego:

```
export default function Page() {
  return (
    <main>
      <Navbar />
      <Hero />
      <About />
      <Services />
      <Gallery />
      <Testimonials />
      <Contact />
      <Footer />
      <WhatsAppFloating />
    </main>
  );
}
```

Cada `<Componente />` es una sección definida en su propio archivo. Esto hace el código modular y fácil de leer.

7. Sistema de rutas de Next.js

Next.js convierte **carpetas en direcciones (URLs)** automáticamente. Cada carpeta con un `page.tsx` es una página:

Archivo	Dirección (URL)
src/app/page.tsx	/
src/app/reservar/page.tsx	/reservar
src/app/admin/page.tsx	/admin
src/app/admin/login/page.tsx	/admin/login
src/app/admin/services/page.tsx	/admin/services
src/app/admin/reservations/page.tsx	/admin/reservations
src/app/admin/records/page.tsx	/admin/records
src/app/admin/reports/page.tsx	/admin/reports

No hay que configurar nada: crear la carpeta y el archivo es suficiente para tener una nueva página.

8. Capa de conexión a Firebase (firebase.ts)

Archivo `src/lib/firebase.ts`

Este archivo **inicializa** Firebase una sola vez y entrega dos objetos que el resto del proyecto usa: `auth` (login) y `db` (base de datos).

```
// arma la configuración leyendo las llaves del .env.local
const firebaseConfig = {
  apiKey: process.env.NEXT_PUBLIC_FIREBASE_API_KEY,
  authDomain: process.env.NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN,
  projectId: process.env.NEXT_PUBLIC_FIREBASE_PROJECT_ID,
  /* ...las demás llaves... */
};

// evita inicializar dos veces (importante con la recarga automática)
const app = getApps().length ? getApp() : initializeApp(firebaseConfig);

export const auth = getAuth(app); // servicio de login
export const db = getFirestore(app); // base de datos
```

Detalle técnico: `getApps().length ? getApp() : initializeApp(...)` es un patrón "singleton": si Firebase ya fue inicializado, lo reutiliza; si no, lo crea. Esto evita errores durante el desarrollo, cuando el código se recarga muchas veces.

9. Capa de datos: servicios (services.ts)

Archivo `src/lib/services.ts`

Contiene todo lo necesario para trabajar con los servicios en Firestore.

El tipo de dato

```
export type Service = {
  id: string;           // identificador único
  category: string;     // Cabello, Uñas, Spa & Estética
  name: string;
  desc: string;
  price: number;
  order?: number;      // el "?" = campo opcional
  active?: boolean;    // true = visible, false = archivado
};
```

Leer servicios

```
// trae TODOS (para el panel admin)
export async function fetchServices() {
  const snap = await getDocs(collection(db, "services"));
  return snap.docs.map((d) => ({ id: d.id, ...d.data() }));
}

// trae solo los ACTIVOS (para el sitio público)
export async function fetchActiveServices() {
  const q = query(collection(db, "services"), where("active", "==", true));
  const snap = await getDocs(q);
  return snap.docs.map((d) => ({ id: d.id, ...d.data() }));
}
```

Conceptos: `collection(db, "services")` apunta a la colección; `getDocs` descarga los documentos; `query(..., where("active", "==", true))` filtra solo los activos. La palabra `await` significa "espera a que termine antes de seguir" (porque pedir datos toma tiempo).

Crear, editar y archivar

```
export async function addService(data) { return addDoc(collection(db,"services"),
{active:true, ...data}); }
export async function updateService(id, data) { return updateDoc(doc(db,"services",id),
data); }
export async function archiveService(id) { return updateDoc(doc(db,"services",id),
{active:false}); } // borrado lógico
export async function restoreService(id) { return updateDoc(doc(db,"services",id),
{active:true}); }
```

Borrado lógico: `archiveService` no elimina el documento; solo cambia `active` a `false`. Así el dato se conserva para los reportes.

Agrupar por categoría

Convierte la lista plana de servicios en grupos por categoría, usando bucles simples:

```
export function groupByCategory(services) {
  const grupos = [];
  for (const s of services) {
    let grupo = grupos.find((g) => g.category === s.category);
    if (!grupo) { grupo = { category: s.category, items: [] }; grupos.push(grupo); }
    grupo.items.push(s);
  }
  return grupos;
}
```

10. Capa de datos: registro diario (records.ts)

Archivo `src/lib/records.ts`

Maneja la bitácora de servicios realizados por día.

```
export type ServiceRecord = {
  id: string; date: string; serviceName: string;
  category: string; price: number; quantity: number; note?: string;
};

// fecha de hoy en formato AAAA-MM-DD
export function todayStr() { return new Date().toLocaleDateString("en-CA"); }

export async function addRecord(data) { return addDoc(collection(db,"records"), data); }

// registros de un día exacto
export async function fetchRecordsByDate(date) {
  const q = query(collection(db,"records"), where("date","==",date));
  return (await getDocs(q)).docs.map(d => ({ id:d.id, ...d.data() }));
}

// registros entre dos fechas (para los reportes)
export async function fetchRecordsBetween(from, to) {
  const q = query(collection(db,"records"), where("date",">=",from), where("date","<=",to));
  return (await getDocs(q)).docs.map(d => ({ id:d.id, ...d.data() }));
}
```

Truco de las fechas: se guardan como texto `AAAA-MM-DD` (ej: "2026-07-08"). En ese formato, comparar fechas como texto (`>=`, `<=`) funciona igual que comparar fechas reales, lo que permite filtrar por rango sin complicaciones.

11. Capa de datos: reservas y control de cupos (booking.ts)

Archivo `src/lib/booking.ts`

Es la parte más elaborada: maneja reservas y evita que se reserve de más.

Generar las horas disponibles

A partir del horario del salón (definido en `site.ts`), calcula las franjas de un día:

```
export function slotsForDate(dateStr) {
  const diaSemana = new Date(dateStr + "T00:00:00").getDay(); // 0=domingo
  const horario = HOURS[diaSemana];
  if (!horario) return []; // cerrado
  const horas = [];
  let minuto = horaAMinutos(horario.open); // ej: 8:00 → 480
  const cierre = horaAMinutos(horario.close);
  while (minuto + SLOT_MIN <= cierre) { // SLOT_MIN = 60
    horas.push(minutosAHora(minuto)); // 480 → "08:00"
    minuto += SLOT_MIN;
  }
  return horas;
}
```

Crear una reserva sin doble-reservar

Se usa una colección aparte, `slots`, con un **contador** por hora. Antes de guardar, se revisa ese contador:

```
export async function createReservation(data) {
  const id = data.date + "_" + data.time; // ej: "2026-07-10_14:00"
  const slotRef = doc(db, "slots", id);

  // 1) revisar cuántos cupos hay ocupados en esa hora
  const slotSnap = await getDoc(slotRef);
  const ocupados = slotSnap.exists() ? slotSnap.data().count : 0;
  if (ocupados >= CAPACITY) throw new Error("FULL"); // CAPACITY = 2

  // 2) guardar la reserva como "pendiente"
  await addDoc(collection(db, "reservations"), { ...data, slotId: id, status: "pendiente" });

  // 3) sumar 1 al contador de esa hora
  await setDoc(slotRef, { date: data.date, time: data.time, count: ocupados + 1 }, {
    merge: true });
}
```

¿Por qué una colección de contadores y no contar reservas? Porque las reglas de seguridad no dejan que el público lea las reservas de otras clientas (privacidad). El contador

de `slots` solo guarda un número, sin datos personales, así que sí puede ser público para mostrar la disponibilidad.

Cambiar estado y liberar cupos

```
export async function setReservationStatus(id, nuevoEstado) {
  const reserva = (await getDoc(doc(db,"reservations",id))).data();
  await updateDoc(doc(db,"reservations",id), { status: nuevoEstado });

  const estabaActiva = reserva.status !== "cancelada";
  const quedaActiva = nuevoEstado !== "cancelada";
  if (estabaActiva && !quedaActiva) cambiarContador(reserva.slotId, -1); // canceló → libera
  if (!estabaActiva && quedaActiva) cambiarContador(reserva.slotId, +1); // reactivó → ocupa
}
```


12. Modelo de datos en Firestore

Cuatro colecciones. Cada documento es una ficha con estos campos:

services

Campo	Tipo	Descripción
category	texto	Categoría del servicio
name / desc	texto	Nombre y descripción
price	número	Precio en colones
order	número	Orden de aparición
active	booleano	true=visible, false=archivado

reservations

Campo	Tipo	Descripción
date / time	texto	Fecha y hora de la cita
serviceName / category	texto	Copia del servicio
price	número	Precio al reservar
customerName / customerPhone	texto	Datos de la clienta
status	texto	pendiente / confirmada / completada / cancelada
slotId	texto	Enlace al contador de la hora

slots

Campo	Tipo	Descripción
date / time	texto	Hora exacta
count	número	Cupos ocupados en esa hora

records

Campo	Tipo	Descripción
date	texto	Día del servicio
serviceName / category	texto	Copia del servicio
price / quantity	número	Precio y cantidad

note	texto	Nota opcional
------	-------	---------------

Nota de diseño (desnormalización): las reservas y registros guardan una **copia** del nombre y precio del servicio, en vez de solo un enlace. Así, aunque el servicio original cambie o se archive, los datos históricos permanecen exactos. Es una decisión intencional para que los reportes sean confiables.

13. Autenticación y protección de rutas

Archivos `src/app/admin/login/page.tsx` y cada página de `admin/`

Iniciar sesión

```
import { signInWithEmailAndPassword } from "firebase/auth";
import { auth } from "@lib/firebase";

await signInWithEmailAndPassword(auth, email, password);
// si el correo/contraseña son válidos, entra; si no, lanza un error
```

Proteger las páginas del panel

Cada pantalla del panel comprueba si hay una sesión activa. Si no, redirige al login:

```
useEffect(() => {
  const unsub = onAuthStateChanged(auth, (usuario) => {
    if (!usuario) router.replace("/admin/login"); // sin sesión → afuera
    else setChecking(false); // con sesión → muestra
  });
  return () => unsub(); // deja de escuchar al salir
}, []);
```

Conceptos: `onAuthStateChanged` "escucha" el estado de sesión y avisa cuando cambia. `useEffect` ejecuta esto al abrir la pantalla. `router.replace` cambia de página. La función devuelta (`unsub`) se ejecuta al salir, para dejar de escuchar y no consumir recursos.

Las contraseñas nunca pasan por el código propio ni se guardan en Firestore: las gestiona Firebase Authentication.

14. Componentes del sitio público

Carpeta `src/components/sections/`

Cada sección de la home es un componente independiente. La más técnica es **Services**, porque carga datos de Firebase:

```
// services.tsx (resumido)
const [groups, setGroups] = useState(fallbackGroups); // datos de respaldo

useEffect(() => {
  fetchActiveServices().then((lista) => {
    if (lista.length > 0) setGroups(groupByCategory(lista));
  }).catch(() => { /* si falla, se queda el respaldo */ });
}, []);
```

Patrón importante: arranca con una lista de respaldo (escrita en `site.ts`) y, cuando Firebase responde, la reemplaza por los datos reales. Si Firebase falla, la página igual muestra algo. Esto se llama "degradación elegante".

Componente	Técnica que usa
navbar.tsx	Estado para el menú móvil; detecta scroll con useEffect.
hero.tsx	Animaciones de entrada con Framer Motion.
gallery.tsx	Estado para abrir/cerrar la imagen ampliada (modal).
testimonials.tsx	Carrusel automático con un temporizador (setInterval).

15. Componentes reutilizables

Componente	Qué hace
<code>service-picker.tsx</code>	Combobox personalizado para elegir un servicio, agrupado por categoría.
<code>whatsapp-icon.tsx</code>	El logo de WhatsApp como imagen SVG (para los botones).
<code>section-shell.tsx</code>	"Molde" con título y descripción que reutilizan las secciones.
<code>utils.ts</code>	Función <code>cn()</code> que junta clases de CSS condicionalmente.

Detalle técnico: cerrar el combobox al hacer clic afuera

El `service-picker` usa una **referencia** al elemento y escucha los clics del documento:

```
const boxRef = useRef(null); // referencia al recuadro

useEffect(() => {
  function alClicAfuera(e) {
    if (boxRef.current && !boxRef.current.contains(e.target)) setOpen(false);
  }
  document.addEventListener("mousedown", alClicAfuera);
  return () => document.removeEventListener("mousedown", alClicAfuera);
}, []);
```

Cómo funciona: `useRef` guarda una referencia al recuadro del combobox. Cada clic en la página se revisa: si fue **fuera** del recuadro, se cierra. Al desmontar el componente, se quita el "escucha" para no dejar código corriendo de más.

16. Módulos del panel de administración

Todos siguen el mismo patrón técnico:

1. **Guardia de sesión** con `onAuthStateChanged` (capítulo 13).
2. **Estado** (`useState`) para los datos y los campos del formulario.
3. **Carga de datos** con `useEffect` al abrir la pantalla, llamando a funciones de `src/lib`.
4. **Acciones** (crear, editar, cambiar estado) que llaman a `src/lib` y vuelven a cargar la lista.

Servicios (services/page.tsx)

Un formulario que sirve para crear y editar (si hay un `editingId`, guarda cambios; si no, crea). Lista los activos y, aparte, los archivados con opción de restaurar.

Reservas (reservations/page.tsx)

Muestra las reservas de un día y botones para cambiar el estado. Cada botón llama a `setReservationStatus`, que además ajusta el contador de cupos.

Registro diario (records/page.tsx)

Formulario para anotar servicios realizados. Calcula en vivo el total del día sumando `precio × cantidad` con un bucle simple.

Reportes (reports/page.tsx)

Pide los registros de un rango de fechas y calcula los totales en el momento:

```
let totalRevenue = 0, totalCount = 0;
for (const r of records) {
  totalRevenue += r.price * r.quantity;
  totalCount   += r.quantity;
}
// luego agrupa por categoría con otro bucle
```

17. Sistema de estilos (Tailwind y CSS global)

El diseño combina dos técnicas:

- **Clases de Tailwind** directamente en los elementos, para estilos puntuales. Ej:
`class="rounded-2xl bg-black/60 px-4 py-3"`.
- **Clases propias** definidas en `src/app/globals.css` para estilos repetidos y complejos.

Variables de color (globals.css)

Los colores se definen una vez como variables y se reutilizan en todo el sitio:

```
:root {  
  --bg: 10 10 10;           /* fondo negro */  
  --fg: 245 240 230;        /* texto crema */  
  --accent: 201 168 76;     /* dorado principal */  
}
```

Y clases propias como `.btn-primary` (botón dorado), `.glass` (efecto vidrio) o `.h2` (título con degradado) se usan en muchas pantallas para mantener consistencia.

18. Reglas de seguridad de Firestore

Archivo `firestore.rules`

Firestore ejecuta estas reglas en el servidor **antes** de permitir cualquier lectura o escritura. Ni siquiera modificando el código del navegador se pueden saltar.

```
match /services/{doc} {
  allow read: if true; // cualquiera lee
  allow write: if request.auth != null; // solo con sesión escribe
}
match /records/{doc} {
  allow read, write: if request.auth != null; // solo admins
}
match /reservations/{doc} {
  allow read, update, delete: if request.auth != null; // admins gestionan
  allow create: if request.resource.data.status == 'pendiente'; // clienta crea
}
match /slots/{doc} {
  allow read, write: if true; // solo un contador, sin datos personales
}
```

Explicación: `request.auth != null` significa "hay un usuario con sesión iniciada". La regla de `reservations` permite a una clienta **crear** una reserva pendiente, pero solo un admin puede **leer** o **editar** (protege los datos personales).

19. Ciclo de vida de un dato (flujo técnico)

Ejemplo técnico: qué ocurre cuando una clienta reserva una cita.

1. La clienta llena el formulario en /reservar (componente React).
2. Al enviar, el componente llama a `createReservation(datos)` [src/lib/booking.ts].
3. `createReservation` usa el SDK de Firebase:
 - a. `getDoc(slots/fecha_hora)` → lee el contador de cupos
 - b. si `count >= CAPACITY` → lanza "FULL" y avisa
 - c. `addDoc(reservations, {...})` → crea la reserva "pendiente"
 - d. `setDoc(slots/..., count + 1)` → suma un cupo
4. Firestore aplica las reglas de seguridad y guarda los datos.
5. El admin abre /admin/reservations:
 - `onAuthStateChanged` confirma la sesión
 - `fetchReservationsByDate(hoy)` descarga las reservas del día
 - la pantalla se dibuja con esos datos (`useState`)
6. El admin cambia el estado → `setReservationStatus(id, "confirmada")`
 - `updateDoc` actualiza el documento en Firestore.

Todo el recorrido respeta las capas: UI → src/lib → Firebase. La UI nunca llama al SDK de Firebase directamente.

20. Glosario técnico

Término	Significado
Framework	Conjunto de herramientas y reglas que dan una estructura lista para construir (ej: Next.js).
Librería	Código ya hecho que se reutiliza (ej: React, Firebase).
SDK	"Kit" de funciones que provee un servicio para conectarse a él (ej: el SDK de Firebase).
Componente	Pieza reutilizable de interfaz en React.
Hook	Función especial de React (useState, useEffect, useRef).
Estado (state)	Datos que cambian y redibujan la pantalla.
Props	Parámetros que recibe un componente.
async / await	Forma de esperar operaciones que tardan (como pedir datos) sin congelar la página.
Promesa (Promise)	Un resultado que "llegará después"; se espera con await.
Query	Consulta a la base de datos con condiciones (ej: solo los activos).
Documento / Colección	Ficha individual / grupo de fichas en Firestore.
Variable de entorno	Valor de configuración externo al código (las llaves en .env.local).
Singleton	Patrón para tener una única instancia de algo (Firebase se inicializa una sola vez).
Desnormalización	Guardar copias de datos a propósito para conservar el histórico.

21. Cómo extender el proyecto (agregar un módulo)

Gracias a la arquitectura por capas, agregar una función nueva sigue siempre los mismos pasos. Ejemplo: agregar un módulo de "Productos".

1. **Capa de datos:** crear `src/lib/products.ts` con el tipo `Product` y funciones `fetchProducts`, `addProduct`, etc. (copiando el patrón de `services.ts`).
2. **Página:** crear `src/app/admin/products/page.tsx` con la guardia de sesión, el estado y el formulario (copiando el patrón de otro módulo).
3. **Acceso:** agregar una tarjeta en el panel `src/app/admin/page.tsx` que enlace a la nueva página.
4. **Seguridad:** agregar la regla de la colección `products` en `firestore.rules` y publicarla.

Como cada módulo existente sigue este mismo molde, el proyecto es fácil de crecer y mantener.

Fin de la documentación técnica

Sistema Web — Gaby Salón y Spa